**PYTHON ACTIVITY BASED LEARNING**

**SHAIK.HANIFA**

-

**CSM3**

**ROLL.NO:26001021**

**PROGRAM-1**

**University Scholarship Allocation: Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.**

**1.Problem Solving Approach**

Identify the inputs, processing requirements, and outputs.

- Input: Student name, aggregate marks, subject-wise marks, attendance percentage, and annual family income.
- Processing: Check eligibility, identify rejected applicants, rank eligible students, and resolve ties.
- Output: List of eligible students in rank order and list of rejected applicants.

**2. Eligibility criteria**

For this example, assume the university uses the following rules.

| **Criterion** | **Requirement** |
|---|---|
| Aggregate marks | At least 75% |
| Mathematics | At least 60 marks |
| Physics | At least 50 marks |
| Chemistry | At least 50 marks |
| Attendance | At least 80% |
| Annual family income | At most ₹3,00,000 |

**Tie-breaking rules:**

1. Higher aggregate marks rank first.
2. If aggregate marks are equal, higher attendance ranks first.
3. If attendance is also equal, lower annual family income ranks first.
4. If all three are equal, alphabetical order of the student's name decides the rank.

These are illustrative rules; an actual university should use its officially approved criteria.

3.Algorithm

1. Start.
2. Read the number of applicants.
3. Store each applicant's name, aggregate marks, subject marks, attendance, and annual family income.
4. Check all eligibility conditions.
5. If every condition is satisfied, add the applicant to the eligible list.
6. Sort eligible applicants by aggregate marks in descending order.
7. Resolve ties using attendance in descending order, income in ascending order, and name in alphabetical order.
8. Display the ranked eligible applicants.
9. Stop.

4.Psudocode

```
BEGIN

  eligible_students = empty list

  INPUT number_of_applicants

  FOR each applicant:
    INPUT name
    INPUT aggregate_marks
    INPUT mathematics_marks
    INPUT physics_marks
    INPUT chemistry_marks
    INPUT attendance
    INPUT annual_income
```

```
        IF aggregate_marks >= 75 AND
           mathematics_marks >= 60 AND
           physics_marks >= 50 AND
           chemistry_marks >= 50 AND
           attendance >= 80 AND
           annual_income <= 300000 THEN

            ADD applicant to eligible_students

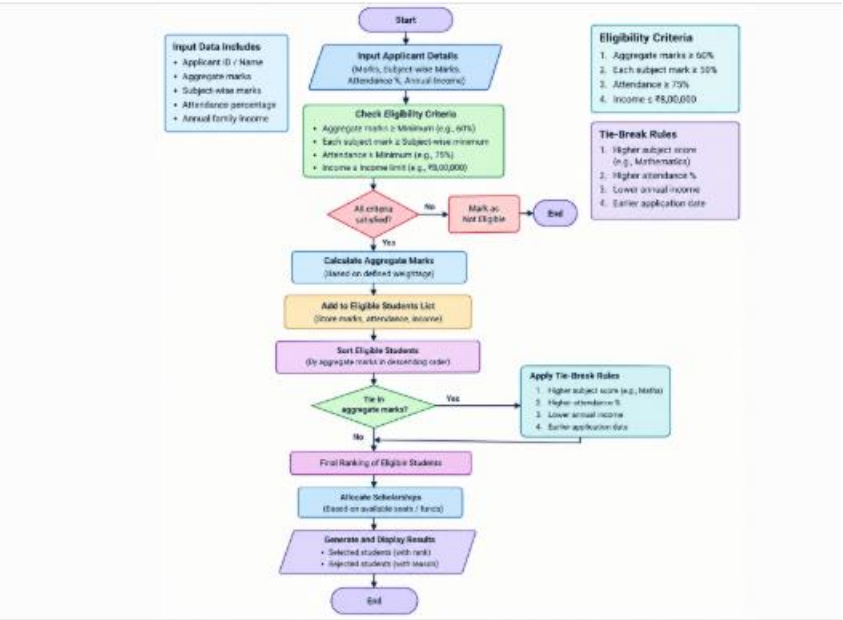
        END IF
    END FOR

    SORT eligible_students BY:
        aggregate_marks descending
        attendance descending
        annual_income ascending
        name ascending

    DISPLAY eligible_students with ranks

END
```

5.Flowchart




1. **Python implementation**

The following program demonstrates all the main concepts in the question.

python

```
# University Scholarship Allocation System

MIN_AGGREGATE = 75
MIN_MATHS = 60
MIN_PHYSICS = 50
MIN_CHEMISTRY = 50
MIN_ATTENDANCE = 80
MAX_INCOME = 300000

# Nested list: each applicant is represented by a list
applicants = [
    ["Ananya", 90, 85, 80, 82, 95, 200000],
    ["Bharat", 90, 85, 80, 82, 90, 180000],
    ["Charan", 78, 65, 55, 60, 85, 250000],
    ["Divya", 88, 70, 65, 68, 92, 350000],
```

```
    ["Esha", 82, 55, 70, 65, 90, 150000],
    ["Farhan", 85, 75, 70, 72, 88, 220000],
]

eligible = []
rejected = []

# Check eligibility using a loop and conditional statements
for student in applicants:
    name, aggregate, maths, physics, chemistry, attendance, income = student

    if (aggregate >= MIN_AGGREGATE
            and maths >= MIN_MATHS
            and physics >= MIN_PHYSICS
            and chemistry >= MIN_CHEMISTRY
            and attendance >= MIN_ATTENDANCE
            and income <= MAX_INCOME):

        eligible.append(student)
    else:
        rejected.append(name)

# Tuple operations and sorting
# Sort by aggregate descending, attendance descending,
# income ascending, and name alphabetically.
eligible.sort(key=lambda s: (-s[1], -s[5], s[6], s[0]))

# Display results using string formatting
print("SCHOLARSHIP RANK LIST")
print("-" * 75)

for rank, student in enumerate(eligible, start=1):
```

```
    name, aggregate, maths, physics, chemistry, attendance, income = student

    print(
        f"Rank {rank}: {name} | "
        f"Aggregate: {aggregate}% | "
        f"Attendance: {attendance}% | "
        f"Income: Rs. {income}"
    )

print("\nRejected applicants:", ", ".join(rejected))
```

**8. Expected output**

text

```
SCHOLARSHIP RANK LIST
---------------------------------------------------------------------------
Rank 1: Ananya | Aggregate: 90% | Attendance: 95% | Income: Rs. 200000
Rank 2: Bharat | Aggregate: 90% | Attendance: 90% | Income: Rs. 180000
Rank 3: Farhan | Aggregate: 85% | Attendance: 88% | Income: Rs. 220000
Rank 4: Charan | Aggregate: 78% | Attendance: 85% | Income: Rs. 250000

Rejected applicants: Divya, Esha
```